JAVASCRIPT · FUNDAMENTOS · ARRAYS Y FUNCIONES

Arrays, objetos y funciones

Una guía para aprender a guardar varios datos, recorrerlos, transformarlos y organizar el código en funciones pequeñas y reutilizables. Pensada como teoría, referencia rápida y apoyo directo para programar.

arrays · for...of · push / pop · filter / map · objetos · funciones · parámetros · return

01 Qué es un array

Cuando necesitamos guardar varios valores relacionados, un array nos permite tenerlos dentro de una única variable.

Muchos datos | **Un array** | **Acceder / recorrer / modificar**

Idea mental útil

Piensa en un array como una fila de cajas numeradas. Cada caja tiene una posición, y la primera posición es la **0**.

JavaScript

```
const lenguajes = ["HTML", "CSS", "JavaScript", "PHP"];

console.log(lenguajes[0]); // HTML
console.log(lenguajes[2]); // JavaScript
```

02

Posiciones y length

La propiedad `length` indica cuántos elementos tiene el array.

Elemento	Posición
"HTML"	0
"CSS"	1
"JavaScript"	2
"PHP"	3

JavaScript

```
console.log(lenguajes.length); // 4

console.log(lenguajes[lenguajes.length - 1]); // PHP
```

Error típico

Si el array tiene 4 elementos, la última posición no es 4: es 3. La numeración empieza en 0.

03

Añadir y eliminar elementos

Los métodos más importantes al empezar son `push()`, `pop()`, `shift()` y `unshift()`.

push()

Añade al final.

pop()

Elimina el último.

unshift()

Añade al principio.

shift()

Elimina el primero.

JavaScript

```
const tareas = ["Estudiar", "Comprar"];

tareas.push("Entrenar");
console.log(tareas);

tareas.pop();
console.log(tareas);
```

04 Recorrer un array

Una de las operaciones más habituales es hacer algo con cada elemento.

JavaScript · for...of

```
const alumnos = ["Ane", "Mikel", "Jon"];

for (const alumno of alumnos) {
    console.log("Alumno: " + alumno);
}
```

Cuándo usarlo

Si solo necesitas leer todos los elementos uno a uno, **for...of** suele ser la opción más fácil de entender.

05

Buscar y filtrar

Con arrays aparecen operaciones muy habituales: comprobar si existe un valor o quedarse solo con algunos elementos.

includes()

Comprueba si un valor existe en el array.

filter()

Crea un nuevo array con los elementos que cumplen una condición.

JavaScript

```
const notas = [3, 7, 5, 9, 4, 8];

console.log(notas.includes(9)); // true

const aprobados = notas.filter((nota) ⇒ nota ≥ 5);
console.log(aprobados); // [7, 5, 9, 8]
```

06

Transformar con map()

`map()` crea un nuevo array transformando cada elemento.

[1, 2, 3] **multiplicar por 2** **[2, 4, 6]**

JavaScript

```
const numeros = [1, 2, 3];

const dobles = numeros.map((numero) ⟹ numero * 2);

console.log(dobles);
```

Diferencia clave

filter() decide qué elementos pasan. **map()** mantiene todos los elementos, pero los transforma.

07

Objetos: varios datos sobre una misma cosa

Un objeto sirve para guardar propiedades relacionadas.

JavaScript

```
const alumno = {
    nombre: "Ane",
    curso: "DAW",
    nota: 8
};

console.log(alumno.nombre);
console.log(alumno.nota);
```

Idea mental útil

Un array suele representar una **lista**. Un objeto suele representar una **cosa** con varias características.

08

Arrays de objetos

Podemos combinar ambas ideas para trabajar con listas de alumnos, productos, videojuegos, reservas, etc.

JavaScript

```
const productos = [
    { nombre: "Ratón", precio: 20, stock: 4 },
    { nombre: "Teclado", precio: 50, stock: 0 },
    { nombre: "Monitor", precio: 180, stock: 3 }
];

for (const producto of productos) {
    console.log(producto.nombre + " - " + producto.precio + "€");
}
```

09

Qué es una función

Una función es un bloque de código al que damos un nombre para poder utilizarlo cuando lo necesitemos.

Datos de entrada **Función** **Resultado**

JavaScript

```
function saludar() {
    console.log("Hola");
}

saludar();
```

Idea mental útil

Crear una función es como fabricar una pequeña herramienta. Después puedes utilizarla tantas veces como necesites.

10 Parámetros

Los parámetros permiten que la misma función trabaje con datos diferentes.

JavaScript

```
function saludar(nombre) {
    console.log("Hola " + nombre);
}

saludar("Ane");
saludar("Mikel");
```

Resultado

La función es la misma. Lo único que cambia es el dato que recibe.

11 return: devolver un resultado

Una función no siempre debe mostrar algo por consola. Muchas veces interesa que calcule un valor y lo devuelva.

JavaScript

```
function sumar(a, b) {
    return a + b;
}

const resultado = sumar(4, 7);
console.log(resultado);
```

No confundas

`console.log()` muestra un valor. `return` devuelve un valor para poder seguir utilizándolo.

12

Funciones que deciden

Una función puede combinar parámetros, condicionales y `return`.

JavaScript

```
function esMayorDeEdad(edad) {
    if (edad ≥ 18) {
        return true;
    }

    return false;
}
```

Versión más corta

```
function esMayorDeEdad(edad) {
    return edad ≥ 18;
}
```

13 Funciones + arrays

Aquí es donde ambas partes empiezan a tener sentido juntas.

JavaScript

```
function contarAprobados(notas) {
    let contador = 0;

    for (const nota of notas) {
        if (nota ≥ 5) {
            contador++;
        }
    }

    return contador;
}

const notas = [3, 7, 5, 9, 2, 8];
console.log(contarAprobados(notas));
```

14

Arrow functions

JavaScript permite escribir funciones de una forma más compacta. Conviene entender primero la forma tradicional.

Función tradicional

```
function doble(n) {
    return n * 2;
}
```

Arrow function

```
const doble = (n) ⇒ {
    return n * 2;
};
```

Por ahora

No necesitas convertir todas tus funciones a arrow functions. Lo importante es entender **qué recibe la función, qué hace y qué devuelve**.

15

Método para resolver ejercicios

Antes de escribir código, divide el problema en pasos pequeños.

1 · Datos ¿Qué información tengo? ¿Es un valor, un array o un objeto?

2 · Objetivo ¿Qué resultado tengo que conseguir?

3 · Recorrido Si hay un array, ¿necesito recorrer todos sus elementos?

4 · Decisión ¿Tengo que comprobar alguna condición?

5 · Función ¿Puedo separar una parte del problema en una función?

6 · Prueba Prueba con varios valores, no solo con el caso que esperas que funcione.

16 Ejercicios · Arrays y objetos

Todos los ejercicios de esta parte requieren programar. Empiezan muy guiados y poco a poco mezclan más conceptos.

1 Tu lista de videojuegos

Crea un array llamado `juegos` con al menos 5 videojuegos. Muestra el array completo, el primero, el tercero y el último.

> **Objetivo:** practicar creación, posiciones y `length`.

▶ **Pista**

2 Lista de la compra

Parte de:

```
let compra = ["pan", "leche", "huevos"];
```

Añade **arroz** y **agua**, muestra el array, elimina el último producto y vuelve a mostrarlo.

▶ **Pista**

3 Lista de alumnos

Recorre este array y muestra `Alumno: nombre` para cada persona.

```
const alumnos = ["Ane", "Mikel", "Jon", "Lucía", "Iker"];
```

▶ **Pista**

4 Suma y media de notas

Con `[7, 5, 8, 9, 6]`, calcula primero la suma total y después la media.

> **Salida esperada:** Total: 35 · Media: 7

▶ **Pista**

5 Buscar aprobados

De `[3, 7, 5, 9, 4, 8, 2]`, crea un nuevo array que contenga solo las notas aprobadas.

> **Resultado:** [7, 5, 9, 8]

▶ **Pista**

6 Aplicar un descuento

Una tienda tiene los precios `[100, 50, 20, 80]`. Crea un nuevo array aplicando un 10 % de descuento a cada precio.

> **Resultado:** [90, 45, 18, 72]

▶ **Pista**

7 Buscar un usuario

Con `["admin", "ana", "mikel", "lucia"]`, pide un nombre mediante `prompt()` y muestra si existe o no en la lista.

▶ **Pista**

8 Nuestro primer objeto

Crea un objeto `alumno` con las propiedades nombre, edad, curso y nota. Después muestra una frase usando sus datos.

> **Ejemplo:** Ane estudia DAW y tiene una nota de 8.

9 Productos disponibles

Usa este array:

```
const productos = [
    { nombre: "Ratón", precio: 20, stock: 4 },
    { nombre: "Teclado", precio: 50, stock: 0 },
    { nombre: "Monitor", precio: 180, stock: 3 },
    { nombre: "Auriculares", precio: 70, stock: 0 }
];
```

Muestra únicamente los productos cuyo stock sea mayor que 0.

▶ **Pista**

10 Mini tienda

Con el array anterior, muestra solo los productos que tengan stock y cuesten menos de 100 €. Al final indica cuántos cumplen ambas condiciones.

Practica: array + objetos + condiciones + filter() + recorrido.

17 Ejercicios · Funciones

La idea es que el código vaya quedando cada vez más organizado y reutilizable.

1 Mi primera función

Crea una función `saludar()` que muestre **¡Hola!**. Después llámala tres veces.

2 Saludar a una persona

Modifica la función anterior para que reciba un nombre como parámetro.

```
saludar("Ane");
saludar("Mikel");
```

> Debe mostrar: Hola Ane · Hola Mikel

3 Sumar dos números

Crea una función `sumar(a, b)` que devuelva la suma con `return`.

```
const resultado = sumar(4, 7);
console.log(resultado);
```

> **Resultado:** 11

4 ¿Es mayor de edad?

Crea `esMayorDeEdad(edad)`. Debe devolver `true` si la edad es 18 o más y `false` en caso contrario.

5 Calculadora

Crea cuatro funciones: `sumar`, `restar`, `multiplicar` y `dividir`. Pide dos números al usuario y utiliza las cuatro funciones.

Objetivo: dividir un problema en funciones pequeñas.

6 Precio con IVA

Crea `calcularIVA(precio)` para devolver el precio con un 21 % de IVA. Como ampliación, permite recibir el porcentaje como segundo parámetro.

▶ **Pista**

7 ¿Es par?

Crea `esPar(numero)` y haz que devuelva un booleano.

▶ **Pista**

8 Nota más alta

Con `[5, 8, 4, 10, 7, 6]`, crea `notaMasAlta(notas)` y devuelve la nota mayor.

▶ **Pista**

9 Contar aprobados

Crea `contarAprobados(notas)`. Para `[3, 7, 5, 9, 2, 8]` debe devolver **4**.

Practica: funciones + arrays + bucles + condicionales.

10 Estadísticas de una clase

Con `[7, 4, 9, 5, 8, 3, 10]`, crea estas funciones:

```
calcularMedia(notas)
contarAprobados(notas)
contarSuspensos(notas)
notaMasAlta(notas)
notaMasBaja(notas)
```

Después muestra un resumen completo de la clase.

> **Salida aproximada:**
> Media: 6.57
> Aprobados: 5
> Suspensos: 2
> Nota más alta: 10
> Nota más baja: 3

18

Reto final · Gestor de notas

Combina arrays, objetos, bucles, condicionales y funciones en un único programa.

Datos iniciales

```
const alumnos = [
    { nombre: "Ane", nota: 8 },
    { nombre: "Jon", nota: 4 },
    { nombre: "Mikel", nota: 7 },
    { nombre: "Lucía", nota: 9 },
    { nombre: "Iker", nota: 3 }
];
```

Crea funciones para:

1.

Mostrar todos los alumnos.

2.

Mostrar únicamente los aprobados.

3.

Mostrar únicamente los suspensos.

4.

Buscar un alumno por nombre.

5.

Calcular la nota media.

6.

Encontrar al alumno con mejor nota.

Objetivo final

Que el programa principal sea corto porque la mayor parte del trabajo esté organizada dentro de funciones.

19 Chuleta rápida

Referencia mínima para tener a mano mientras programas.

Quiero...	Uso...
Crear una lista	`const datos = [ ... ]`
Saber cuántos elementos hay	`array.length`
Añadir al final	`push()`
Eliminar el último	`pop()`
Recorrer todos	`for ... of`
Comprobar si existe	`includes()`
Quedarme con algunos	`filter()`
Transformar todos	`map()`
Crear una función	`function nombre( ... ) { ... }`
Devolver un resultado	`return`